Fixed-arity nondeterministic remainder programs: expanded proof and executable checks

Research note accompanying the uniform-process construction

October 16, 2026

Abstract

The Euclidean-prefix construction gives a family of finite nondeterministic recursive programs with arbitrarily small logarithmic remainder-call coefficients. Each program uses only remainder and the zero test, and its arity is fixed independently of the inputs. This does not give a single sublogarithmic program. We give the program schema, correctness and cost arguments, and an executable compilation into first-order syntax. Lean checks the fixed-width recurrence and the asymptotic program bound. It also checks translations preserving remainder counts into finite first-order syntax and into a specialized two-stack recursive machine, correctness of every terminating syntactic evaluation, and existence of a successful evaluation on every input.

1 The result and its quantifiers

We use the nondeterministic convention of Moschovakis: every convergent computation must return the correct value, divergent computations are disregarded, and remainder complexity is the minimum number of remainder calls in a convergent computation [1, §2E, (2E-2), (2E-5)]. All logarithms below have base 2.

Theorem 1. *For each integer $p \geq 1$, there is one finite nondeterministic recursive program $P_p = P_W$, $W = 4^p + 5$ (defined in Section 2), over $(\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0)$ computing the gcd such that*

$$\limsup_{a\to\infty} \max_{1\leq b\leq a} \frac{c_{P_p}(a,b)}{\log a} \leq \frac{3}{p}.$$

Its generator has $W = 4^p + 5$ individual parameters. In particular,

$$\forall \varepsilon > 0 \ \exists P, B \ \forall a \geq b \geq 1, \qquad c_P(a,b) \leq \varepsilon \log a + B.$$

Lean: `Syntax.eventually_eval_bound` (first assertion) and `Syntax.arbitrarily_small_program` (second assertion); see Section 6.

The program may depend on ε, but not on a, b. This is compatible with the Main Conjecture for algorithms, that is, recursive programs, deterministic or nondeterministic [1, p. 2]: its lower-bound coefficient may depend on the program. The companion paper leaves that conjecture open; it refutes only the uniform-process version (4F-5). The bound concerns remainder calls, not internal recursive-machine steps or the time needed to find a successful branch.

The theorem and its proof also appear in the companion paper's section "Fixed programs with arbitrarily small coefficients" [2]; this note retains the expanded operational discussion and the executable checks. The accompanying `model-novelty-audit.txt` records the model comparison and the scope of the primary-literature search.

2 A finite program for each fixed width

Fix $W \geq 2$ and write $\vec{x} = (x_0, \ldots, x_{W-1})$. (Only $W \geq 2$ is needed below, matching Lean's $2 \leq w$; the cost bound uses $W = 4^p + 5$.) Define the following finite family of equations:

$$\begin{aligned}
\mathrm{choose}(\vec{x}) &= x_i && (0 \leq i < W),\\
\mathrm{set}_i(\vec{x}, z) &= \mathrm{gen}(x_0, \ldots, x_{i-1}, z, x_{i+1}, \ldots, x_{W-1}) && (0 \leq i < W),\\
\mathrm{gen}(\vec{x}) &= \mathrm{choose}(\vec{x}),\\
\mathrm{gen}(\vec{x}) &= \mathrm{set}_i(\vec{x}, \mathrm{rem}(\mathrm{choose}(\vec{x}), \mathrm{choose}(\vec{x}))) && (0 \leq i < W),\\
\mathrm{gen}(\vec{x}) &= \mathrm{set}_i(\vec{x}, \mathrm{gen}(\vec{x})) && (0 \leq i < W),\\
\mathrm{gen}(\vec{x}) &= \mathrm{gen}(\mathrm{choose}(\vec{x}), \ldots, \mathrm{choose}(\vec{x})).
\end{aligned}$$

There are exactly W occurrences of choose in the last equation. The setters set_i have arity $W + 1$, the largest auxiliary arity in this presentation; inlining them, as in the companion paper and in Lean, gives maximal arity $\max(W, 3)$, the 3 coming from accept below. The companion paper takes $W \geq 3$, so that this is W. The repeated definitions express finite nondeterministic choice. Each index i is a syntactic index: after fixing W, all the displayed families are expanded into finitely many ordinary equations. The symbols W, i are not runtime data, and there is no array-access, list-construction, or numeric-constant primitive.

Call-by-value meaning. The last equation copies or permutes any available parameters without a remainder call. In the subcomputation equation $\mathrm{set}_i(\vec{x}, \mathrm{gen}(\vec{x}))$, call-by-value evaluates the inner generator once and retains the caller's parameters until its return. The result replaces one parameter, and the outer generator continues. These are ordinary first-order calls; each recursive variable has fixed arity [1, §§2A, 2D].

W bounds arity, not storage. The recursion stack may grow, so W is not a bound on the total number of values in the entire computation.

Verification wrapper. Complete the program by setting $\mathrm{loop}(z) = \mathrm{loop}(z)$ and

$$\mathrm{accept}(a, b, g) = \begin{cases} \mathrm{loop}(g), & \mathrm{eq}_0(g),\\ g, & \mathrm{eq}_0(\mathrm{rem}(a, g)) \text{ and } \mathrm{eq}_0(\mathrm{rem}(b, g)),\\ \mathrm{loop}(g), & \text{otherwise.} \end{cases}$$

The cases are nested conditionals: the zero test is performed before either division, and the conjunction is expanded into two conditionals. Finally set

$$P_W(a, b) = \begin{cases} a, & \mathrm{eq}_0(b),\\ \mathrm{accept}(a, b, \mathrm{gen}(a, b, a, \ldots, a)), & \text{otherwise.} \end{cases}$$

Only rem and eq_0 are primitive symbols in this program. On a successful computation with $b > 0$, verification adds exactly two remainder calls and the whole program makes four zero tests.

Presentations. The Lean presentation inlines the setters: for example, the remainder alternative becomes $\mathrm{gen}(\vec{x}) = \mathrm{gen}(x_0, \ldots, x_{i-1}, \mathrm{rem}(\mathrm{choose}(\vec{x}), \mathrm{choose}(\vec{x})), x_{i+1}, \ldots, x_{W-1})$, and the subcomputation alternative is inlined in the same way. The resulting program has a fixed signature of four internal symbols, choose, gen, accept, and loop, named `pick`, `gen`, `accept`, `loop` in Lean's `Syntax.Fn`. The executable Python presentation retains the setters.

3 Correctness and the operational interpretation

Proposition 2. *For every $W \geq 2$, every convergent computation of $P_W(a, b)$ returns $\gcd(a, b)$, and at least one such computation exists on every natural input. Positive scaling preserves the generator's successful choice sequences and remainder-call counts.*

Proof. Induct on a finite generator evaluation. Every common divisor of its input parameters divides its output: this is immediate for a selected parameter, is preserved by remainder and copying, and passes through a recursive subcomputation and its continuation.

Consequently $\gcd(a, b)$ divides every candidate g generated at the root. A successful check requires $g > 0$, $g \mid a$, and $g \mid b$, so also $g \mid \gcd(a, b)$. Thus $g = \gcd(a, b)$. The separate zero-input branch is correct as well.

There is always a successful generator computation following Euclid. With the current pair in the first two parameters, overwrite the first by its remainder modulo the second, then swap and duplicate parameters to obtain the next pair. When the second is zero, return the first. The choice sequence is selected in the existence proof; no additional test is introduced into the generator. Finally $\mathrm{rem}(gx, gy) = g\,\mathrm{rem}(x, y)$ for $g > 0$, and nonzero divisors remain nonzero under scaling. Induction preserves the same computation and its cost. □

Correctness of every terminating evaluation and existence of a successful one are checked as `Syntax.eval_head_correct` and `Syntax.eval_head_exists`.

The program is pure first-order syntax: variables, remainder, zero-test conditionals, and calls to a fixed finite signature, each symbol having a finite set of bodies. Its call-by-value evaluation trees evaluate all arguments before the chosen body and sum their remainder counts. For these pure terms the evaluation trees correspond, with the same result and remainder count, to the book's recursive machine. In the forward direction, induct on a tree and serialize its argument evaluations right to left before entering the chosen body. Transition locality embeds each evaluation in its caller context [1, Table 1 and Lemma 2D.1, pp. 75–76]. A remainder node adds one remainder call; a zero-test conditional adds none and evaluates only its chosen branch. Conversely, decompose a terminating machine computation into its completed argument computations and selected body or branch, and induct on its length. Each selector occurrence makes its own choice, with no shared mutable state. The setters in this note merely pass their evaluated arguments to the generator, so inlining them also preserves results and remainder counts. Lean checks this correspondence; see Section 6.

4 The fixed-prefix cost bound

Proof of Theorem 1. Write $r_0 = a$, $r_1 = b$, $r_{i+1} = r_{i-1} \bmod r_i$ for the Euclidean remainders of a coprime pair. Fix $p \geq 1$; thus $W = 4^p + 5$. We use the Euclidean-prefix lemma and the three-call contraction lemma of the companion paper's section "Three-call contractions" [2]. Put

$$k = 2^p, \qquad h = k^2 + 1 = 4^p + 1, \qquad W = h + 4.$$

The entire prefix $r_0, \ldots, r_h$ fits in $h + 1$ generator parameters. Three further parameters suffice for the current modulus and two scratch residues. Each contraction uses at most three remainder calls and can overwrite the scratch residues, leaving the prefix available.

For precision, let $T_W(n)$ be the maximum, over $a, b \leq 2^n$, of the minimum generator remainder count needed to produce $\gcd(a, b)$ from the initial tuple $(a, b, a, \ldots, a)$. Euclid gives $T_W(n) = O(n)$,

and copying gives symmetry. Positive scaling lets us analyze coprime inputs and transport the resulting computations to general inputs without extra calls.

Fix $0 < \beta < 1/2$ and set $\alpha = 1 - \beta/2$. Use the early threshold N^α and the target modulus N^β, with $N = 2^n$. In the *early case*, some positive r_i with $1 \le i \le h$ is at most N^α; it is treated below. Otherwise $r_h > N^\alpha$, so $2a/r_h < 2N^{1-\alpha} < N^\beta$ for sufficiently large n, since $1 - \alpha = \beta/2 < \beta$. The prefix therefore occupies at least $k+1$ residue classes modulo every modulus used before stopping. A contraction decreases the modulus by a factor greater than k, and $J = \lceil n/p \rceil$ contractions suffice.

After reaching a small modulus m, store $u = a \bmod m$ and $v = b \bmod m$ as the two scratch residues. Use a recursive generator subcomputation to produce $d = \gcd(m, u)$, first routing its parameters to copies of m, u. Its caller retains v. On return, replace the modulus parameter by d, then route to copies of d, v for the second gcd computation. The identity

$$\gcd(\gcd(m, a \bmod m), b \bmod m) = \gcd(m, \gcd(a, b))$$

recovers the original gcd, since every generated modulus is divisible by it. Only one result crosses each subcomputation boundary. In the early case, route directly to the consecutive Euclidean pair formed by the first such r_i and its successor, at a cost of at most h remainder calls. Thus the early branch and the two recursive gcd computations of the companion paper's construction are realizable in one fixed generator of width W, with the same remainder counts.

It follows that

$$T_W(n) \le \max\left\{T_W(\lceil \alpha n \rceil) + h + 1, \quad 2T_W(\lceil \beta n \rceil) + h + 3\lceil n/p \rceil + 2\right\}.$$

All thresholds and operand choices select witnesses among computations of the same fixed program; β is not a program parameter.

Write $L_W = \limsup_{n\to\infty} T_W(n)/n$, finite and nonnegative by Euclid. Here h is fixed, so dividing the recurrence by n and taking upper limits gives

$$L_W \le \max\{\alpha L_W,\ 2\beta L_W + 3/p\}.$$

If the first term is the larger, $L_W \le \alpha L_W$ forces $L_W = 0$, since $\alpha < 1$; otherwise $(1 - 2\beta)L_W \le 3/p$. Letting $\beta \downarrow 0$ yields $L_W \le 3/p$. Taking $n = \lceil \log a \rceil$ and adding the two verification calls proves the first assertion of Theorem 1. For the second, choose $p > 3/\varepsilon$ and absorb the finitely many smaller inputs in B. □

This argument uses a fixed h, unlike the companion paper's sublogarithmic construction. The price for a smaller logarithmic coefficient is a larger finite program. Neither the stated coefficient nor the parameter count is claimed optimal.

5 What still prevents the direct sublogarithmic compilation

The companion paper's sublogarithmic construction uses a prefix length $h = \lceil \sqrt{n} \rceil$ that grows with the input. Replacing W by a function of the input in the compiler would produce different programs on different inputs.

Saving a growing prefix in nested calls also does not directly implement the stored prefix. A current function body can use its individual parameters; values retained in older continuations are not an indexed store. Returning exposes an older continuation but passes only the return value from the completed call. For example, a recursive prefix builder that carries only the current

Euclidean pair leaves the innermost call without all earlier remainders as parameters. The three-call contraction lemma cannot be applied there to a stored prefix as though those values were freely available.

Regenerating the whole prefix for every contraction is too expensive for this construction: a positive prefix $r_0, \ldots, r_h$ requires $h - 1$ remainder calls, so rebuilding it for each of J' contractions costs $(h-1)J'$ calls in addition to the contractions. In that construction h is of order $\sqrt{n}$ and the number J' of contractions is of order $n/\log n$, so this is of order $n^{3/2}/\log n$.

This diagnoses the direct implementations, not every possible recursive program. A different organization of the computations might avoid the need for a fully reusable growing prefix. No impossibility theorem for such an organization has been proved here. In particular, the quantifier exchange

$$\forall \varepsilon\ \exists P_\varepsilon \qquad \text{to} \qquad \exists P\ \forall \varepsilon$$

remains unjustified, and this note does not give a single sublogarithmic program. The separate note *Growing prefixes in a fixed remainder program* [3] tests a fixed program that regenerates selected prefix operands, analyzes its accounting limitation, and identifies a possible structural direction.

6 Formal verification

Lean declarations named below are in the namespace `ShortGCD.RegisterProgram`, and we omit that prefix. They are part of the normal `lake build` target.

Generator runs. The relation `Run` has four constructors: return a parameter, overwrite one parameter by a remainder, route parameters, or run a subcomputation and overwrite one parameter by its result. These correspond to the four generator alternatives of the program above. For a remainder step, the two selector calls choose its operands; for a routing step, the W selector calls choose its argument tuple. Internal selector calls make no remainder calls.

First-order syntax. `Syntax.Fn` is the signature and `Syntax.Eval` the evaluation-tree semantics of Section 3. The theorem `Syntax.run_invokes` translates every generator run into an evaluation of the inlined first-order generator, preserving its remainder count exactly. `Syntax.accepts_eval` includes the verification wrapper. Independently, `Syntax.eval_head_correct` proves correctness of *every* terminating evaluation, and `Syntax.eval_head_exists` proves existence of a successful evaluation on every input. Thus correctness is not restricted to the selected fast witnesses.

Two-stack machine. The module `ShortGCD/RegisterSmallStep.lean` gives an explicit two-stack machine for this language: it evaluates arguments right to left and has separate transitions for zero tests and branching. Its closures encode substituted source terms, not a new data type available to the program. `Syntax.Machine.eval_iff_steps` checks the tree-to-machine equivalence in both directions, and `Syntax.Machine.Step.context` checks transition locality. The machine specializes the book's transition table to the fixed language and signature, rather than formalizing arbitrary structures and vocabularies. The equivalence with the Python presentation retaining setters remains a mathematical argument.

Main declarations.

- `maxCost_step`: the fixed-width finite recurrence;

- `Fixed.limsup_profile_le`: the leading coefficient $3/p$;
- `Fixed.eventually_accepts_bound`: the bound for accepted runs, including the final verification;
- `Syntax.eventually_eval_bound`: the same bound for first-order program evaluations;
- `Syntax.arbitrarily_small_program`: the quantified $\varepsilon \log a + B_\varepsilon$ statement with the program fixed before the inputs are chosen;
- `Syntax.Machine.arbitrarily_small_program`: the same quantified bound directly for finite machine transition sequences.

Axiom audit. The research entry point `research/RegisterProgram.lean`, run from the repository root by

```
lake env lean research/RegisterProgram.lean
```

prints the axiom dependencies of these declarations, recorded in `register-axioms.txt`; only the standard axioms `propext`, `Classical.choice`, and `Quot.sound` occur.

7 Experiments

`register_program.py` compiles a fixed-width instance to an AST containing only individual variables, function calls, and conditionals. A static checker checks arities and sorts and rejects extra primitives and numeric literals. A separate evaluator implements call-by-value, with right-to-left argument evaluation. It executes the compiled syntax and counts actual remainder calls. The offline planner supplies nondeterministic choices; its unrestricted arithmetic is not executed by the program. The Python compiler and interpreter are independently checked executable artifacts, not verified implementations of the Lean semantics.

Each compiled program ($W = 9, 21, 69$) is run on the same 46 inputs, for 138 successful cases in all, including zero, unordered, scaled, random, and Fibonacci inputs. The suite additionally replays 129 positive-input cases after scaling both inputs by 13 with exactly the same choices. It exercises 2,086 contractions and 101 recursive subcomputations, with nesting depth up to four. Eight negative checks cover rejected candidates, zero divisors, malformed choices, and invalid program syntax. A two-parameter instance separately replays Euclid.

On one seeded 2,048-bit pair, whose ordinary Euclidean computation takes 1,188 remainder calls, the fixed programs give:

p	W	Remainder calls	Internal calls
1	9	1,036	4,476
2	21	713	3,564
3	69	634	4,920

The remainder counts include final verification. "Internal calls" counts the calls of the program's own function symbols (the entry function, choose, the setters, gen, accept) in the executed Python presentation; these calls are not charged, and the remainder calls in their bodies are counted in the previous column. These are successful computations, not evidence of optimal costs or a deterministic method of finding the choices. Fibonacci examples are also recorded, but should not

be taken as evidence of a new lower-bound separation: Pratt's program is already known to be fast on that family [1, Corollary 6C.7].

Run from the repository root:

```
python3 research/check_register_program.py
python3 -O research/check_register_program.py
python3 research/register_program.py --width 9
```

References

[1] Yiannis N. Moschovakis. *Abstract Recursion and Intrinsic Complexity*. Lecture Notes in Logic 48, Cambridge University Press / ASL, 2019. doi:10.1017/9781108234238.

[2] Jason Davies. *The Euclidean algorithm is not optimal among uniform processes*. Companion repository manuscript, October 2026.

[3] *Growing prefixes in a fixed remainder program*. Research note accompanying the uniform-process construction, `research/single-program-investigation.pdf`, October 2026.